SO – TP

Trabalho Prático – Simulador de Concorrência com Monitorização

LSIRC

Licenciatura em Segurança Informática e Redes de Computadores

SO

Sistemas Operativos

Diogo Costa - 8240696

Hugo Martins – 8230273

ESTG | P.PORTO

2025/26

Índice

1 Introdução 4
1.1 Importância da concorrência em sistemas operativos 5
1.2 Introdução ao eBPF 5
2 Problemas de Concorrência 6
2.1 Explicação dos problemas 6
2.2 Ilustração com a Implementação Prática 6
2.3 Exploração em Sistemas Operativos Reais (Cibersegurança) 7
3 Monitorização ao Estilo eBPF 8
3.1 Arquitetura da Classe de Monitorização 8
3.2 Métricas e Deteção de Anomalias 9
3.2.1 Deteção de Deadlocks (Introspeção) 9
3.2.2 Deteção de Starvation (Análise Temporal) 10
3.2.3 Estatísticas de Acesso 10
3.2.4 Nota sobre a Deteção de Race Conditions 10
3.3 Monitorização, Proteção e Mitigação 11
4 Implementação 11
4.1 Estrutura do Código 11
4.2 Descrição da Simulação e Soluções 11
4.2.1 Mitigação de Race Conditions 12
4.2.2 Prevenção de Deadlocks 13
4.2.3 Mitigação de Starvation 14
4.3 Logs Reais da Simulação 15
4.3.1 Race Condition (Ataque à Integridade) 15
4.3.2 Deadlock (Ataque DoS) 17
4.3.3 Starvation (Bloqueio de Auditoria) 18
5 Conclusão 20
5.1 Principais Aprendizagens 20
5.2 Transposição para Implementação Real com eBPF 20

Índice de Figuras

Figura 1 - Arquitetura Singleton da classe SecurityMonitor 8
Figura 2 - Utilização da ThreadMXBean para deteção de Deadlocks 9
Figura 3 - Algoritmo para deteção de Starvation 10
Figura 4 - Comparação: Acesso vulnerável vs. Proteção com Monitor 12
Figura 5 - Prevenção de Deadlocks 13
Figura 6 - Implementação do modo justo no Acesso a Ficheiros 14

1 Introdução

O presente trabalho foi desenvolvido no âmbito da unidade curricular de Sistemas Operativos, pertencente ao plano de estudos da Licenciatura em Segurança Informática em Redes de Computadores (LSIRC), lecionada no ano letivo 2025/2026. O principal objetivo consistiu no aprofundamento dos conhecimentos
sobre multiprocessamento, comunicação e sincronização de processos, através da implementação prática de um simulador de concorrência e de um sistema de monitorização inspirado na tecnologia eBPF(extended Berkeley Packet Filter).

Nos sistemas operativos modernos, a capacidade de executar múltiplas tarefas simultaneamente é essencial, mas introduz desafios complexos na gestão de recursos partilhados. A ausência de mecanismos de sincronização adequados não resulta apenas em erros de execução, mas cria vulnerabilidades críticas de segurança. Fenómenos como Race Conditions, Deadlocks e Starvation podem ser explorados maliciosamente para provocar corrupção de dados, elevação de privilégios ou ataques de Negação de Serviço.

O foco deste trabalho não é apenas replicar estes problemas teóricos, mas sim analisá-los sob uma perspetiva de cibersegurança. O projeto visa demonstrar como a execução coordenada de threads é vital em ambientes de produção e como a falta de controlo pode levar ao colapso de serviços críticos ou à inconsistência de dados sensíveis.

Como componente inovadora, o trabalho inclui o desenvolvimento de um mecanismo de monitorização inspirado no eBPF. Tal como o eBPF permite estender as funcionalidades do kernel para observabilidade e segurança sem alterar o código-fonte do sistema operativo, a solução desenvolvida implementa uma classe de monitorização que rastreia eventos em tempo real, deteta anomalias comportamentais e gera alertas de segurança, tudo isto num ambiente simulado em Java.

Para a realização deste projeto, foi utilizada a linguagem de programação Java, recorrendo às suas bibliotecas nativas de concorrência e gestão de threads. A implementação prática foca-se na criação de cenários vulneráveis, na sua exploração como vetores de ataque e, posteriormente, na aplicação das devidas correções utilizando primitivas de sincronização, demonstrando a importância de uma programação
segura e auditável.

1.1 Importância da concorrência em sistemas operativos

A concorrência é um pilar fundamental dos sistemas operativos modernos. Permite que múltiplas tarefas (processos ou threads) progridam simultaneamente, maximizando a utilização da CPU e garantindo a responsividade do sistema.

Num ambiente multiprogramado, a gestão eficiente de recursos partilhados é crítica; sem ela, o sistema tornar-se-ia lento e incapaz de realizar operações paralelas, como responder a um clique do rato enquanto processa dados em segundo plano.

1.2 Introdução ao eBPF

O eBPF (extended Berkeley Packet Filter) é uma tecnologia revolucionária que permite executar programas num ambiente "sandbox" dentro do kernel do Linux, sem a necessidade de alterar o código-fonte do kernel ou carregar módulos adicionais.

1. **Para que serve:** Originalmente criado para filtrar pacotes de rede, evoluiu para uma ferramenta poderosa de observabilidade, segurança e performance.
2. **Como monitoriza:** O eBPF funciona através de hooks (ganchos) em eventos do sistema, como chamadas ao sistema (syscalls), pontos de entrada/saída de funções e eventos de rede.
3. **Relevância para a Cibersegurança:** Permite uma visibilidade profunda e segura. Ao contrário de módulos de kernel tradicionais que podem crashar o sistema, o eBPF é verificado antes de correr, garantindo que não bloqueia o SO. É usado para detetar anomalias, intrusões e ataques em tempo real com um impacto mínimo no desempenho.

2 Problemas de Concorrência

2.1 Explicação dos problemas

No desenvolvimento de sistemas concorrentes, a ausência de sincronização adequada leva a três problemas clássicos:

Race Condition (Condição de Corrida): Ocorre quando o comportamento do software depende da sequência ou temporização incontrolável de eventos (como a ordem de execução de threads). Se duas threads tentarem modificar o mesmo dado simultaneamente, o resultado pode ser inconsistente.

Deadlock (Bloqueio Mútuo): É uma situação em que dois ou mais processos ficam bloqueados indefinidamente, pois cada um detém um recurso que o outro necessita, criando um ciclo de dependência impossível de resolver sem intervenção externa.

Starvation (Míngua): Acontece quando um processo ou thread, embora pronto para executar, é perpetuamente negado o acesso aos recursos necessários devido a políticas de escalonamento injustas (ex: processos de alta prioridade monopolizam a CPU).

2.2 Ilustração com a Implementação Prática

Na simulação desenvolvida, estes problemas foram replicados da seguinte forma:

1. **Race Condition:** Na classe CarteiraDigital, o método adicionarSaldoInseguro simula aleitura e escrita de um saldo partilhado. Duas threads ("Hacker-1" e "Hacker-2") tentam adicionar 100€ simultaneamente. Devido à falta de exclusão mútua, o valor final é frequentemente 100€ em vez de 200€, demonstrando perda de dados.
2. **Deadlock:** Na classe ServidorBaseDeDados, o método processarTransacao força uma thread a segurar o recurso A e pedir o B, enquanto outra segura o B e pede o A. Isto cria uma espera circular detetada pelo Monitor.
3. **Starvation:** Na classe SistemaFicheiros, configurámos um acesso "Injusto" (Unfair). Threads "VIP" de alta prioridade entram num loop contínuo de leitura, impedindo que a thread de "Auditoria" (baixa prioridade) consiga adquirir o lock, simulando o bloqueio de um serviço de segurança.

2.3 Exploração em Sistemas Operativos Reais (Cibersegurança)

Estes problemas técnicos traduzem-se diretamente em vulnerabilidades graves em sistemas reais :

Race Conditions (Corrupção e Privilégios): Em sistemas operativos, uma race condition clássica é a vulnerabilidade **TOCTOU** (Time-of-Check to Time-of-Use). Um atacante pode explorar o intervalo de tempo entre a verificação de permissões de um ficheiro (Check) e a sua utilização (Use) para substituir o ficheiro original por um link simbólico para um ficheiroprotegido (ex:/etc/passwd), conseguindo assim aumentar os seus privilégios ou corromper dados do sistema.

Deadlocks (Denial of Service - DoS): Um atacante pode intencionalmente desencadear um deadlock num servidor web ou base de dados. Ao enviar pedidos especificamente desenhados para bloquear recursos críticos de forma cruzada, o atacante consegue congelar o sistema, impedindo que utilizadores legítimos acedam ao serviço. Isto constitui um ataque de Negação de Serviço (DoS) eficaz, pois não requer largura de banda elevada, apenas lógica maliciosa.

Starvation (Bloqueio de Serviços Críticos): O starvation pode ser usado para cegar sistemas de defesa. Um malware pode gerar tráfego de I/O (Input/Output) massivo com prioridade elevada, impedindo que processos de auditoria, antivírus ou sistemas de registo de logs consigam aceder ao disco rígido. Se o serviço de auditoria entrar em starvation, as atividades maliciosas do atacante não serão registadas.

3 Monitorização ao Estilo eBPF

3.1 Arquitetura da Classe de Monitorização

A classe SecurityMonitor foi desenvolvida para funcionar como o núcleo central de observabilidade do sistema. A sua arquitetura segue o padrão de desenho Singleton, garantindo a existência de uma única instância responsável pela recolha, agregação e análise de eventos gerados pelas diferentes threads em execução.

Esta centralização é fundamental para simular o comportamento do eBPF em sistemas Linux, que observa o sistema a partir de um ponto privilegiado no kernel, de forma transversal e desacoplada da lógica da aplicação. De forma análoga, o SecurityMonitor não interfere com a execução funcional do sistema, limitando-se à observação e registo de eventos relevantes.

Para garantir desempenho e evitar que o próprio mecanismo de monitorização introduza condições de corrida, são utilizadas estruturas de dados thread-safe, nomeadamente o ConcurrentHashMap, assegurando a integridade dos dados recolhidos em ambientes altamente concorrentes.

```java
public class SecurityMonitor {  22 usages

    private static SecurityMonitor instancia;  3 usages
    private BufferedWriter escritor;  7 usages
    private final String FICHEIRO_LOG = "logs.txt";  1 usage

    // Mapa para estatísticas
    private final ConcurrentHashMap<String, Integer> estatisticasThread = new ConcurrentHashMap<>();

    private SecurityMonitor() {  1 usage
        try {
            escritor = new BufferedWriter(new FileWriter(FICHEIRO_LOG,  append: true));
            registarEvento( nomeThread: "SISTEMA",  mensagem: "Monitor de segurança iniciado.");
        } catch (IOException e) {
            System.err.println("Erro ao inicializar log: " + e.getMessage());
        }
    }

    public static synchronized SecurityMonitor obterInstancia() {  6 usages
        if (instancia == null) {
            instancia = new SecurityMonitor();
        }
        return instancia;
    }
```

Figura 1 - Arquitetura Singleton da classe SecurityMonitor

3.2 Métricas e Deteção de Anomalias

O mecanismo de monitorização recolhe métricas em tempo de execução e recorre a técnicas de introspeção da Máquina Virtual Java (JVM) para identificar padrões anómalos de concorrência que possam indiciar falhas de segurança.

3.2.1 Deteção de Deadlocks (Introspeção)

Para a deteção de situações de bloqueio mútuo (deadlock), o monitor utiliza a API de gestão da JVM, concretamente a classe ThreadMXBean. Esta abordagem funciona como uma sonda (probe) que interroga o estado interno da JVM, permitindo identificar ciclos de dependência entre threads bloqueadas.

Sempre que um deadlock é confirmado, o sistema gera um alerta crítico e regista o evento, permitindo a sua análise posterior sob a perspetiva da segurança.

```java
public void detetarDeadlocks() {  1 usage
    ThreadMXBean bean = ManagementFactory.getThreadMXBean();
    long[] idsDeadlock = bean.findDeadlockedThreads();

    if (idsDeadlock != null && idsDeadlock.length > 0) {
        registarEvento( nomeThread: "ALERTA-SEGURANÇA",  mensagem: "========================================");
        registarEvento( nomeThread: "ALERTA-SEGURANÇA",  mensagem: "DEADLOCK DETETADO!");
        registarEvento( nomeThread: "ALERTA-SEGURANÇA",  mensagem: "========================================");

        ThreadInfo[] infos = bean.getThreadInfo(idsDeadlock,  lockedMonitors: true,  lockedSynchronizers: true);
        for (ThreadInfo info : infos) {
            if (info != null) {
                String msg = String.format("Thread '%s' bloqueada por '%s'",
                        info.getThreadName(),
                        info.getLockOwnerName());
                registarEvento( nomeThread: "ALERTA-SEGURANÇA", msg);
            }
        }
    }
}
```

Figura 2 - Utilização da ThreadMXBean para deteção de Deadlocks

Para além da deteção de bloqueios efetivos, a arquitetura do monitor foi concebida de forma extensível, permitindo, numa implementação futura, a deteção de potenciais deadlocks. Tal seria possível através da construção dinâmica de um grafo de alocação de recursos, possibilitando a identificação de ciclos de dependência iminentes antes da ocorrência de um bloqueio definitivo.

3.2.2 Deteção de Starvation (Análise Temporal)

A deteção de starvation baseia-se na análise temporal do acesso aos recursos partilhados. Para cada thread, é calculado o intervalo de tempo (Δ) entre o momento do pedido de acesso a um recurso e a sua obtenção efetiva:

$$\Delta = \text{horaEntrada} - \text{horaChegada}$$

Quando este intervalo ultrapassa um limiar de segurança previamente definido (1000 ms), o sistema considera que a thread se encontra em situação de inanição, o que pode indicar um bloqueio deliberado de serviço.

```java
public void registarEspera(String nomeThread, long tempoEsperaMs) {  3 usages
    final long LIMITE_STARVATION = 1000; // 1 segundo

    if (tempoEsperaMs > LIMITE_STARVATION) {
        registarEvento( nomeThread: "ALERTA-RISCO",  mensagem: "**************************************************");
        registarEvento( nomeThread: "ALERTA-RISCO",  mensagem: "Starvation detetada na thread: " + nomeThread);
        registarEvento( nomeThread: "ALERTA-RISCO",  mensagem: "Tempo de espera excessivo: " + tempoEsperaMs + " ms");
        registarEvento( nomeThread: "ALERTA-RISCO",  mensagem: "**************************************************");
    }
}
```

Figura 3 - Algoritmo para deteção de Starvation

3.2.3 Estatísticas de Acesso

Para além da deteção de anomalias, o monitor mantém estatísticas persistentes relativas ao número de acessos de cada thread aos recursos partilhados. Estas métricas permitem uma análise post-mortem, facilitando a identificação de comportamentos abusivos, tais como a monopolização recorrente de recursospor uma única thread.

3.2.4 Nota sobre a Deteção de Race Conditions

Importa salientar uma distinção conceptual relevante: ao contrário de deadlocks e starvation, que correspondem a estados observáveis durante a execução, as race conditions são fenómenos probabilísticos e frequentemente invisíveis em tempo real.

Assim, o SecurityMonitor não deteta uma race condition pelo simples facto de múltiplas threads acederem concorrentemente a um recurso, mas sim através da análise post-mortem dos efeitos produzidos.

Resultados inconsistentes, como a corrupção de dados ou valores inesperados, são interpretados como indícios claros da ocorrência de uma condição de corrida, permitindo validar a integridade dos dados após a execução de secções críticas.

3.3 Monitorização, Proteção e Mitigação

A implementação do SecurityMonitor transforma a gestão da concorrência numa ferramenta ativa de cibersegurança, aproximando-se do funcionamento de um Sistema de Deteção de Intrusões (IDS).

Ao contrário de uma monitorização meramente passiva, o sistema fornece informação acionável:

- A deteção de um deadlock permite inferir a ocorrência de um ataque de Negação de Serviço (DoS), possibilitando a intervenção administrativa, como o reinício controlado dos serviços afetados.
- A deteção de starvation em threads responsáveis por auditoria ou registo de eventos pode indicar uma tentativa de ocultação de atividade maliciosa através do bloqueio dos mecanismos de logging.

Esta capacidade de observação contínua constitui um elemento essencial da defesa em profundidade em sistemas operativos modernos.

4 Implementação

4.1 Estrutura do Código

O projeto foi organizado em três pacotes lógicos para separar responsabilidades, facilitando a manutenção e a clareza do código:

Monitor: Contém o SecurityMonitor, o cérebro da observabilidade. Esta classe centraliza todos os registos e a lógica de deteção de anomalias (eBPF simulado).

Resources: Contém os recursos partilhados onde residem as vulnerabilidades e as correções:

- CarteiraDigital (Race Conditions);
- ServidorBaseDeDados (Deadlocks);
- SistemaFicheiros (Starvation).

Scenarios: Contém as classes Main (DemoRaceCondition, DemoDeadlock, DemoStarvation) que orquestram a criação de threads e a execução dos ataques simulados.

4.2 Descrição da Simulação e Soluções

A simulação ocorre em duas fases distintas para cada cenário. Primeiro, executa-se o método vulnerável para demonstrar o problema e gerar alertas no monitor. Em seguida, executa-se a versão corrigida para demonstrar a mitigação.

Abaixo ilustram-se as correções aplicadas nos cenários mais críticos, utilizando o código desenvolvido.

4.2.1 Mitigação de Race Conditions

Na CarteiraDigital, a vulnerabilidade ocorria devido à falta de atomicidade na atualização do saldo.A correção passou pela utilização da palavra-chave synchronized, que garante exclusão mútua.

```java
public void adicionarSaldoInseguro(double valor) {  2 usages
    if (valor <= 0) {
        monitor.registarEvento(Thread.currentThread().getName(),  mensagem: "Tentativa de deposito inseguro com valor invalido: " + valor);
        return;
    }

    String nomeThread = Thread.currentThread().getName();
    monitor.registarEvento(nomeThread,  mensagem: " Deposito INSEGURO iniciado: " + valor + "€ (SEM sincronização - risco de race condition)");

    // Secção critica = race condition
    double saldoAtual = saldo;

    try {
        Thread.sleep( millis: 50 + (long)(Math.random() * 100));
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }

    saldo = saldoAtual + valor;

    monitor.registarEvento(nomeThread,  mensagem: " Deposito inseguro concluído. Saldo atual: " + saldo + "€ (possível perda de dados)");
}
public synchronized void adicionarSaldoSeguro(double valor) {  2 usages
    if (valor <= 0) {
        monitor.registarEvento(Thread.currentThread().getName(),  mensagem: "Tentativa de deposito seguro com valor invalido: " + valor);
        return;
    }

    String nomeThread = Thread.currentThread().getName();
    monitor.registarEvento(nomeThread,  mensagem: " Deposito SEGURO iniciado: " + valor + "€ (COM synchronized - exclusão mútua garantida)");

    double saldoAntes = saldo;

    try {
        Thread.sleep( millis: 50 + (long)(Math.random() * 100));
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }

    saldo = saldoAntes + valor;

    monitor.registarEvento(nomeThread,  mensagem: " Deposito seguro concluído. Saldo atual: " + saldo + "€ (integridade garantida)");
}
```

Figura 4 - Comparação: Acesso vulnerável vs. Proteção com Monitor

4.2.2 Prevenção de Deadlocks

No ServidorBaseDeDados, os bloqueios mútuos foram resolvidos implementando uma estratégia de Resource Ordering (Ordenação de Recursos). O código abaixo garante que os recursos são sempre adquiridos numa ordem determinística.

```java
public void processarTransacaoSegura(ServidorBaseDeDados servidorRemoto) {  2 usages
    String thread = Thread.currentThread().getName();

    ServidorBaseDeDados primeiro = this;
    ServidorBaseDeDados segundo  = servidorRemoto;

    if (this.nome.compareTo(servidorRemoto.obterNome()) > 0) {
        primeiro = servidorRemoto;
        segundo  = this;
    }

    monitor.registarEvento(thread,  mensagem: "Resource Ordering aplicado: " + primeiro.obterNome() + " → " + segundo.obterNome() + " (ordem alfabética)");

    synchronized (primeiro) {
        monitor.registarEvento(thread,  mensagem: " " + primeiro.obterNome() + " bloqueado (1° lock adquirido)");

        try { Thread.sleep( millis: 80); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }

        synchronized (segundo) {
            monitor.registarEvento(thread,  mensagem: " " + segundo.obterNome() + " bloqueado (2° lock adquirido - deadlock IMPOSSÍVEL)");
            monitor.registarEvento(thread,  mensagem: " Transação SEGURA concluída entre " + this.nome + " e " + servidorRemoto.obterNome());
        }
    }
}
```

Figura 5 - Prevenção de Deadlocks

4.2.3 Mitigação de Starvation

No SistemaFicheiros, o problema de inanição (Starvation) ocorria porque as threads de alta prioridade (VIP) monopolizavam o acesso. A solução consistiu na implementação de um mecanismo de justiça (fairness) utilizando ReentrantLock.

```java
public class SistemaFicheiros {  3 usages

    private final SecurityMonitor monitor;  9 usages
    private final ReentrantLock chave;  3 usages
    private final String politica;  3 usages
    private final boolean isFair;  2 usages

    public SistemaFicheiros(boolean fair) {  1 usage
        this.monitor = SecurityMonitor.obterInstancia();
        this.politica = fair ? "JUSTO" : "INJUSTO";
        this.chave = new ReentrantLock(fair);
        this.isFair = fair;

        monitor.registarEvento( nomeThread: "SISTEMA",  mensagem: "Sistema de ficheiros iniciado: " + politica);
    }
    *
    * Usa um ReentrantLock com política justa ou injusta.
    * Regista eventos e tempos de espera no SecurityMonitor.
    */
    public void lerFicheiroCritico() {  3 usages
        String nome = Thread.currentThread().getName();
        long chegada = System.currentTimeMillis();

        monitor.registarEvento(nome,  mensagem: "A tentar obter lock do ficheiro [Política: " + politica + "]");

        try {
            chave.lockInterruptibly();

            try {
                long entrada = System.currentTimeMillis();
                long espera = entrada - chegada;

                if (!isFair) {
                    monitor.registarEspera(nome, espera);
                } else if (espera > 3000) {
                    monitor.registarEspera(nome, espera);
                }

                monitor.registarEvento(nome,  mensagem: " Lock adquirido! A aceder ao ficheiro(Tempo de espera: " + espera + "ms)");

                Thread.sleep( millis: 100);

            } finally {
                chave.unlock();
                monitor.registarEvento(nome,  mensagem: " Lock libertado (ficheiro fechado)");
            }

        } catch (InterruptedException e) {
            long tempoPerdido = System.currentTimeMillis() - chegada;

            monitor.registarEvento(nome,  mensagem: " INTERROMPIDO após " + tempoPerdido + "ms de espera (nunca conseguiu o lock)");
            monitor.registarEspera(nome, tempoPerdido);

            Thread.currentThread().interrupt();
        }
    }
```

Figura 6 - Implementação do modo justo no Acesso a Ficheiros

4.3 Logs Reais da Simulação

Abaixo apresentam-se excertos reais gerados pelo ficheiro registo_atividade.txt, validando a deteção de eventos pelo Monitor.

4.3.1 Race Condition (Ataque à Integridade)

Neste cenário, verifica-se a corrupção de dados na Fase 1 (perda de 100€) e a correção total na Fase 2, validada pelo relatório estatístico final.

```
[SISTEMA          ] Monitor de segurança iniciado.
[INFO             ] ═══════════════════════════════════════════════════════════════
[INFO             ] CONCEITO: Race Condition (Condição de Corrida)
[INFO             ] DEFINIÇÃO: Múltiplas threads acedem a dados partilhados
[INFO             ]          sem sincronização, causando resultados imprevisíveis
[INFO             ] ═══════════════════════════════════════════════════════════════
[TESTE-FASE-1     ] DEMONSTRAÇÃO DO PROBLEMA (versão insegura)
[INFO             ] CENÁRIO: 2 threads depositam 100€ simultaneamente
[INFO             ] SALDO INICIAL: 0.0€  |  SALDO ESPERADO: 200.0€
[INFO             ] RESULTADO REAL: Perda de dados devido a race condition
[CARTEIRA         ] Carteira digital criada com saldo inicial de 0.0€
[Hacker-1         ]  Deposito INSEGURO iniciado: 100.0€ (SEM sincronização - risco de race condition)
[Hacker-2         ]  Deposito INSEGURO iniciado: 100.0€ (SEM sincronização - risco de race condition)
[Hacker-1         ]  Deposito inseguro concluído. Saldo atual: 100.0€ (possível perda de dados)
[Hacker-2         ]  Deposito inseguro concluído. Saldo atual: 100.0€ (possível perda de dados)
[RESULTADO        ] ───────────────────────────────────────────────────────────────
[RESULTADO        ] SALDO FINAL INSEGURO: 100.0€
[ALERTA           ]  INCONSISTÊNCIA DETETADA!
[ALERTA           ]  Valor perdido: 100.0€
[ALERTA           ]  CAUSA: Race condition - múltiplas threads leram o mesmo valor
[ALERTA           ]  CONSEQUÊNCIA: Falha grave de integridade dos dados
[RESULTADO        ] ───────────────────────────────────────────────────────────────

============================================================

[TESTE-FASE-2     ] CORREÇÃO DO PROBLEMA (versão segura com synchronized)
[INFO             ] SOLUÇÃO: Modificador 'synchronized' garante exclusão mútua
[INFO             ] GARANTIA: Apenas 1 thread por vez pode executar o método
[INFO             ] RESULTADO ESPERADO: Integridade total dos dados (200.0€)
[CARTEIRA         ] Carteira digital criada com saldo inicial de 0.0€
[Admin-1          ]  Deposito SEGURO iniciado: 100.0€ (COM synchronized - exclusão mútua garantida)
[Admin-1          ]  Deposito seguro concluído. Saldo atual: 100.0€ (integridade garantida)
[Admin-2          ]  Deposito SEGURO iniciado: 100.0€ (COM synchronized - exclusão mútua garantida)
[Admin-2          ]  Deposito seguro concluído. Saldo atual: 200.0€ (integridade garantida)
[RESULTADO        ] ───────────────────────────────────────────────────────────────
[RESULTADO        ] SALDO FINAL SEGURO: 200.0€
[SUCESSO          ]  SUCESSO TOTAL - Integridade dos dados garantida!
[SUCESSO          ]  Synchronized preveniu completamente a race condition
[RESULTADO        ] ───────────────────────────────────────────────────────────────
[RELATÓRIO        ] ========================================
[RELATÓRIO        ] Relatorio final de atividade
[RELATÓRIO        ] ========================================
[RELATORIO        ] Thread: TESTE-FASE-1       ->   1 eventos
[RELATORIO        ] Thread: CARTEIRA           ->   2 eventos
[RELATORIO        ] Thread: Hacker-1           ->   2 eventos
[RELATORIO        ] Thread: TESTE-FASE-2       ->   1 eventos
```

```
[RELATORIO        ] Thread: Hacker-2          ->   2 eventos
[RELATORIO        ] Thread: SUCESSO           ->   2 eventos
[RELATORIO        ] Thread: INFO              ->  11 eventos
[RELATORIO        ] Thread: RESULTADO         ->   6 eventos
[RELATORIO        ] Thread: RELATÓRIO         ->   3 eventos
[RELATORIO        ] Thread: Admin-1           ->   2 eventos
[RELATORIO        ] Thread: Admin-2           ->   2 eventos
[RELATORIO        ] ========================================
[SISTEMA          ] Monitor encerrado.
```

4.3.2 Deadlock (Ataque DoS)

Este registo demonstra a eficácia da Ordenação de Recursos (Fase 1) e a deteção de DoS pelo Monitor na Fase 2.

```
[SISTEMA          ] Monitor de segurança iniciado.
[INFO             ] ═══════════════════════════════════════════════════════════
[INFO             ] CONCEITO: Deadlock (Bloqueio Mútuo)
[INFO             ] DEFINIÇÃO: Duas ou mais threads bloqueadas permanentemente,
[INFO             ]          cada uma esperando por um recurso detido pela outra
[INFO             ] ═══════════════════════════════════════════════════════════
[FASE-1           ] INÍCIO DA SIMULAÇÃO SEGURA (prevenção de deadlock)
[INFO             ] TÉCNICA: Resource Ordering (ordenação alfabética de recursos)
[INFO             ] GARANTIA: Todas as threads adquirem locks na mesma ordem
[INFO             ] RESULTADO ESPERADO: Nenhum deadlock
[SISTEMA          ] Servidor de base de dados 'Servidor-01' inicializado e pronto.
[SISTEMA          ] Servidor de base de dados 'Servidor-02' inicializado e pronto.
[Thread-Segura-1  ] Resource Ordering aplicado: Servidor-01 → Servidor-02 (ordem alfabética)
[Thread-Segura-2  ] Resource Ordering aplicado: Servidor-01 → Servidor-02 (ordem alfabética)
[Thread-Segura-1  ] Servidor-01 bloqueado (1º lock adquirido)
[Thread-Segura-1  ] Servidor-02 bloqueado (2º lock adquirido - deadlock IMPOSSÍVEL)
[Thread-Segura-1  ] Transação SEGURA concluída entre Servidor-01 e Servidor-02
[Thread-Segura-2  ] Servidor-01 bloqueado (1º lock adquirido)
[Thread-Segura-2  ] Servidor-02 bloqueado (2º lock adquirido - deadlock IMPOSSÍVEL)
[Thread-Segura-2  ] Transação SEGURA concluída entre Servidor-02 e Servidor-01
[RESULTADO        ] Fase 1 concluída com SUCESSO - nenhuma thread ficou bloqueada
[RESULTADO        ] Resource Ordering preveniu o deadlock

======================================================================

[FASE-2           ] INÍCIO DA SIMULAÇÃO INSEGURA (provocar deadlock)
[INFO             ] PROBLEMA: Aquisição cruzada de locks
[INFO             ] Thread-Ataque-1: bloqueia Servidor-01 → tenta Servidor-02
[INFO             ] Thread-Ataque-2: bloqueia Servidor-02 → tenta Servidor-01
[INFO             ] RESULTADO ESPERADO: Deadlock permanente (ciclo de espera)
[Thread-Ataque-2  ] Servidor-02 Transação INSEGURA iniciada (bloqueou Servidor-02)
[Thread-Ataque-1  ] Servidor-01 Transação INSEGURA iniciada (bloqueou Servidor-01)
[Thread-Ataque-1  ] Servidor-01 Tentando bloquear Servidor-02 (RISCO: aquisição cruzada de locks)
[Thread-Ataque-2  ] Servidor-02 Tentando bloquear Servidor-01 (RISCO: aquisição cruzada de locks)
[VERIFICAÇÃO      ] ----------------------------------------
[VERIFICAÇÃO      ] Analise do estado das threads em curso...
[ALERTA-SEGURANÇA ] ========================================
[ALERTA-SEGURANÇA ] DEADLOCK DETETADO!
[ALERTA-SEGURANÇA ] ========================================
[ALERTA-SEGURANÇA ] Thread 'Thread-Ataque-1' bloqueada por 'Thread-Ataque-2'
[ALERTA-SEGURANÇA ] Thread 'Thread-Ataque-2' bloqueada por 'Thread-Ataque-1'
[VERIFICAÇÃO      ] ----------------------------------------
[CONCLUSÃO        ] Deadlock detetado – o programa encontra-se bloqueado permanentemente.
```

4.3.3 Starvation (Bloqueio de Auditoria)

Neste log, observa-se a competição entre threads VIP e Auditoria, culminando na apresentação das estatísticas finais recolhidas pelo Monitor.

```
[SISTEMA          ] Monitor de segurança iniciado.
[INFO             ] ═══════════════════════════════════════════════════════════
[INFO             ] CONCEITO: Starvation (Inanição)
[INFO             ] DEFINIÇÃO: Uma thread nunca consegue aceder a um recurso
[INFO             ]          porque outras threads de maior prioridade
[INFO             ]          monopolizam continuamente esse recurso
[INFO             ] ═══════════════════════════════════════════════════════════
[TESTE-FASE-1       ] DEMONSTRAÇÃO DO PROBLEMA (política INJUSTA)
[INFO             ] CENÁRIO: 20 threads WORKER (prioridade máxima) monopolizam o lock
[INFO             ]         1 thread AUDITORIA (prioridade mínima) nunca consegue aceder
[INFO             ] POLÍTICA: ReentrantLock(fair=false) - SEM garantia de ordem
[INFO             ] RESULTADO ESPERADO: Starvation da thread AUDITORIA
[SISTEMA          ] Sistema de ficheiros iniciado: INJUSTO
[WORKER-1         ] Tentando a obter a chave [INJUSTO]
[WORKER-2         ] Tentando a obter a chave [INJUSTO]
[WORKER-4         ] Tentando a obter a chave [INJUSTO]
[WORKER-3         ] Tentando a obter a chave [INJUSTO]
[WORKER-5         ] Tentando a obter a chave [INJUSTO]
[WORKER-1         ] Acedeu ao ficheiro (Espera: 0ms)
[WORKER-1         ] Libertou o ficheiro.
[WORKER-2         ] Acedeu ao ficheiro (Espera: 103ms)
[WORKER-1         ] Tentando a obter a chave [INJUSTO]
[WORKER-2         ] Libertou o ficheiro.
[WORKER-4         ] Acedeu ao ficheiro (Espera: 204ms)
[WORKER-2         ] Tentando a obter a chave [INJUSTO]
[AUDITORIA        ] Tentando aceder ao lock [INJUSTO]
[ALERTA-RISCO     ] ************************************************
[ALERTA-RISCO     ] Starvation detetada na thread: WORKER-11
[ALERTA-RISCO     ] Tempo de espera excessivo: 1014 ms
[ALERTA-RISCO     ] ************************************************
[WORKER-11        ] Acedeu ao ficheiro (Espera: 1014ms)
[WORKER-10        ] Tentando a obter a chave [INJUSTO]
[WORKER-11        ] Libertou o ficheiro.
[WORKER-11        ] Tentando a obter a chave [INJUSTO]
[ALERTA-RISCO     ] ************************************************
[ALERTA-RISCO     ] Starvation detetada na thread: WORKER-12
[ALERTA-RISCO     ] Tempo de espera excessivo: 1116 ms
[ALERTA-RISCO     ] ************************************************
[WORKER-12        ] Acedeu ao ficheiro (Espera: 1116ms)
[WORKER-12        ] Libertou o ficheiro.
[ALERTA-RISCO     ] ************************************************
[WORKER-12        ] Tentando a obter a chave [INJUSTO]

====================================================================

[TESTE-FASE-2       ] CORREÇÃO DO PROBLEMA (política JUSTA)
[INFO             ] SOLUÇÃO: ReentrantLock(fair=true) - garante ordem FIFO
[INFO             ] GARANTIA: Threads são atendidas por ordem de chegada
[INFO             ] CENÁRIO: Menos threads WORKER (15) + política justa
[INFO             ] RESULTADO ESPERADO: AUDITORIA consegue aceder (sem starvation)
[SISTEMA          ] Sistema de ficheiros iniciado: JUSTO
[WORKER-1         ] Tentando a obter a chave [JUSTO]
```

```
[WORKER-2          ] Tentando a obter a chave [JUSTO]
[WORKER-6          ] Tentando a obter a chave [JUSTO]
[WORKER-3          ] Tentando a obter a chave [JUSTO]
[WORKER-7          ] Tentando a obter a chave [JUSTO]
[WORKER-2          ] Acedeu ao ficheiro (Espera: 103ms)
[WORKER-1          ] Tentando a obter a chave [JUSTO]
[WORKER-2          ] Libertou o ficheiro.
[WORKER-6          ] Acedeu ao ficheiro (Espera: 204ms)
[WORKER-2          ] Tentando a obter a chave [JUSTO]
[AUDITORIA         ] Tentando a obter a chave [JUSTO]
[WORKER-6          ] Libertou o ficheiro.
[WORKER-3          ] Acedeu ao ficheiro (Espera: 304ms)
[WORKER-6          ] Tentando a obter a chave [JUSTO]
[WORKER-7          ] Acedeu ao ficheiro (Espera: 405ms)
[WORKER-3          ] Libertou o ficheiro.
[WORKER-3          ] Tentando a obter a chave [JUSTO]
[WORKER-7          ] Libertou o ficheiro.
[WORKER-4          ] Acedeu ao ficheiro (Espera: 505ms)
[WORKER-7          ] Tentando a obter a chave [JUSTO]
[WORKER-4          ] Libertou o ficheiro.
[WORKER-5          ] Acedeu ao ficheiro (Espera: 606ms)
[WORKER-4          ] Tentando a obter a chave [JUSTO]
[WORKER-14         ] Libertou o ficheiro.
[WORKER-12         ] Acedeu ao ficheiro (Espera: 1503ms)
[WORKER-12         ] Libertou o ficheiro.
[RELATÓRIO         ] ==========================================
[RELATÓRIO         ] Relatorio final de atividade
[RELATÓRIO         ] ==========================================
[RELATORIO         ] Thread: TESTE-FASE-1      ->   1 eventos
[RELATORIO         ] Thread: WORKER-20         ->   6 eventos
[RELATORIO         ] Thread: TESTE-FASE-2      ->   1 eventos
[RELATORIO         ] Thread: INFO              ->   2 eventos
[RELATORIO         ] Thread: RELATÓRIO         ->   3 eventos
[RELATORIO         ] Thread: AUDITORIA         ->   6 eventos
[RELATORIO         ] Thread: WORKER-2          ->  15 eventos
[RELATORIO         ] Thread: WORKER-1          ->  15 eventos
[RELATORIO         ] Thread: WORKER-4          ->  12 eventos
[RELATORIO         ] Thread: WORKER-11         ->  12 eventos
[RELATORIO         ] Thread: WORKER-3          ->  12 eventos
[RELATORIO         ] Thread: WORKER-10         ->  12 eventos
[RELATORIO         ] Thread: WORKER-6          ->  12 eventos
[RELATORIO         ] Thread: WORKER-5          ->  12 eventos
[RELATORIO         ] Thread: WORKER-8          ->  12 eventos
[RELATORIO         ] Thread: WORKER-7          ->  12 eventos
[RELATORIO         ] Thread: WORKER-9          ->  12 eventos
[RELATORIO         ] Thread: RESULTADO         ->   2 eventos
[RELATORIO         ] Thread: WORKER-15         ->  12 eventos
[RELATORIO         ] Thread: WORKER-14         ->  12 eventos
[RELATORIO         ] Thread: WORKER-13         ->  12 eventos
[RELATORIO         ] Thread: WORKER-12         ->  12 eventos
[RELATORIO         ] Thread: WORKER-19         ->   9 eventos
[RELATORIO         ] Thread: WORKER-18         ->   6 eventos
[RELATORIO         ] Thread: WORKER-17         ->   6 eventos
[RELATORIO         ] Thread: WORKER-16         ->   6 eventos
[RELATORIO         ] ==========================================
[SISTEMA           ] Monitor encerrado.
```

5 Conclusão

O presente trabalho prático, desenvolvido no âmbito da unidade curricular de Sistemas Operativos, permitiu alcançar os objetivos propostos, demonstrando a aplicação prática de técnicas de multiprocessamento e a sua relação intrínseca com a cibersegurança. Através da implementação de um simulador em Java, foi possível explorar cenários de condições de corrida, bloqueios mútuos e inanição, cumprindo os requisitos de deteção e mitigação exigidos.

A realização deste projeto consolidou a compreensão de que a gestão de recursos partilhados em sistemas operativos não é apenas uma questão de desempenho, mas um vetor crítico de segurança. A ausência de sincronização adequada não resulta apenas em erros de cálculo; cria vulnerabilidades exploráveis que podem comprometer a integridade e a disponibilidade de serviços críticos.

5.1 Principais Aprendizagens

Em conformidade com os objetivos pedagógicos do trabalho, destacam-se as seguintes aprendizagens fundamentais sobre concorrência e segurança:

1. **Sincronização como Defesa:** Ficou demonstrado que primitivas como synchronized e ReentrantLock são essenciais para prevenir a corrupção de memória e garantir a consistência de dados sensíveis, mitigando ataques baseados em Race Conditions.
2. **Impacto dos Bloqueios na Disponibilidade:** A simulação de Deadlocks evidenciou como a gestão incorreta de recursos pode levar à paragem total do sistema, agindo como um vetor para ataques de Negação de Serviço (DoS). A estratégia de ordenação de recursos (Resour ce Ordering) provou ser uma mitigação eficaz.
3. **Observabilidade e Auditoria:** A implementação do Monitor de Segurança realçou a importância de vigiar o comportamento das threads. A deteção de Starvation em threads de auditoria demonstrou como atacantes podem tentar ocultar a sua atividade explorando prioridades de escalonamento.

5.2 Transposição para Implementação Real com eBPF

Uma componente central deste trabalho foi a simulação da lógica de monitorização inspirada na tecnologia eBPF (extended Berkeley Packet Filter). Embora o simulador tenha sido executado em espaço de utilizador (User Space) sobre a JVM, a lógica é transponível para uma implementação real no núcleo (kernel) do sistema operativo Linux.

Para implementar uma solução equivalente num sistema operativo real, utilizando eBPF, seriam necessárias adaptações técnicas significativas relativamente ao ambiente de execução e à interceção de eventos.

Em primeiro lugar, ao contrário da simulação em Java, o código de monitorização seria escrito em C restrito e compilado para bytecode BPF. Este código seria carregado diretamente no ker-

nel após uma verificação rigorosa de segurança pelo verificador estático do eBPF, garantindo que o programa de monitorização não compromete a estabilidade do sistema operativo, uma garantia que na simulação Java é gerida apenas pelo tratamento de exceções.

Em segundo lugar, os pontos de interceção sofreriam uma alteração de nível. Enquanto no simulador foram instrumentados métodos de objetos Java, numa implementação real utilizar-se-iam kprobes (kernel probes) ou tracepoints anexados a chamadas de sistema e funções de sincronização do próprio kernel, como mutex_lock, mutex_unlock ou schedule.

Por fim, o armazenamento de métricas e estatísticas, gerido no simulador através de estruturas em memória heap como o ConcurrentHashMap, seria implementado utilizando Mapas eBPF (BPF Maps). Estas estruturas de dados especializadas permitem a partilha eficiente de informação entre o código que corre no kernel e as aplicações de monitorização no espaço de utilizador, com um impacto no desempenho (overhead) significativamente menor do que a gestão de memória de uma máquina virtual.

Em suma, este trabalho permitiu simular com sucesso os desafios reais de sistemas multiprogramados. A abordagem de monitorização adotada provou que a observabilidade ativa é uma camada indispensável na arquitetura de segurança de sistemas operativos modernos.